A Hybrid Chess Engine: Combining Classical Search with Neural Network Evaluation

Shubham Kumar Amritasingh Thakur

Foundations of Artificial Intelligence

Northeastern University, Spring 2026

ABSTRACT

We present a two-phase chess engine combining classical adversarial search with a learned neural network (NN) evaluation function. Phase 1 implements Minimax with Alpha-Beta pruning, iterative deepening, quiescence search, and a handcrafted evaluation function covering material, piece-square tables, pawn structure, king safety, mobility, center control, and rook placement. Phase 2 trains a three-layer MLP on 298,000 positions [4]from the Lichess Elite Database labeled by Stockfish 17 at depth 8 [5]. Switching from noisy game-outcome labels to per-position Stockfish evaluations reduces validation loss 8× (0.73 to 0.0895). A 70/30 blend of NN and material counting improves match results from 0/6 to 1.5/6 against the heuristic engine. Our results confirm that explicit domain knowledge outperforms limited-data neural networks, but that label quality and hybrid evaluation close a meaningful portion of that gap.

1. INTRODUCTION

Chess has served as a central benchmark in artificial intelligence since Shannon formalized game-tree search and the minimax strategy in 1950 [1]. Over the following decades, handcrafted engines grew progressively stronger, culminating in Deep Blue's defeat of world champion Garry Kasparov in 1997. The field shifted dramatically when Silver et al. demonstrated that AlphaZero, trained purely through self-play reinforcement learning without any human-designed heuristics, could surpass every prior engine within hours [2]. Stockfish sub-

sequently adopted NNUE (Efficiently Updatable Neural Networks), a hybrid architecture that preserves classical search depth while replacing handcrafted evaluation with a network trained on billions of labeled positions.

These developments raise a practical question for resource-constrained settings: given roughly 300,000 training positions and consumer hardware, how much of the gap between handcrafted heuristics and learned evaluation can be closed, and which design decisions matter most?

Our project constructs a complete, playable chess AI in two phases. Phase 1 implements a classical heuristic engine using Minimax with Alpha-Beta pruning [3], encoding well-established chess principles through a handcrafted evaluation function. Phase 2 replaces that evaluation with a trained MLP, with systematic experiments across labeling strategies and hybrid blending ratios. We evaluate both engines through head-to-head matches, tracking not just win rates but qualitative play patterns. Section 2 describes our methods, Section 3 presents results, and Section 4 discusses findings and future directions.

2. METHODS

2.1. Search Algorithm

Both engines share the same search framework: Minimax with Alpha-Beta pruning [3]. White maximizes the evaluation score; Black minimizes it. Alpha-Beta pruning eliminates branches provably unable to affect the result, reducing the effective branching factor from approximately 35 to $\sqrt{35} \approx 6$ in the best case. Four further optimizations are layered on top. Iterative deepening searches progressively deeper (depth 1, 2, 3, ...) and returns the best result found within the time limit, ensuring a valid move is always available. Move ordering sorts moves as checks first, then captures by MVV-LVA (Most Valuable Victim, Least Valuable Attacker), then threats, then quiet moves, with transposition table hits always searched first to maximize pruning efficiency. Quiescence search extends leaf nodes by continuing to

search captures and checks until the position is quiet, preventing the horizon effect where a significant tactic lies just beyond the search depth. A transposition table caches evaluated positions by FEN key, storing the score, depth, and best move to avoid redundant evaluation of transposed positions.

2.2. Phase 1: Heuristic Evaluation

The evaluation function returns a centipawn score from White's perspective and combines eight components. Material balance uses standard piece values: Pawn = 100, Knight = 320, Bishop = 330, Rook = 500, Queen = 900 centipawns. Piece-square tables (PST) apply per-square bonuses and penalties for each piece type — knights are penalized at the board edges and rewarded centrally; kings use separate middlegame and endgame tables. Pawn structure penalizes doubled and isolated pawns and rewards passed pawns, with bonuses scaling as the pawn advances. King safety rewards castling and a pawn shield in the middlegame; the component is disabled in the endgame, where king activity is an asset. Mobility rewards the side with more legal moves. Center control rewards occupying or attacking d4, d5, e4, e5, and the extended center. Rook placement rewards open and semi-open files and the seventh rank. Bishop pair adds a flat bonus for retaining both bishops. Game phase is inferred from the remaining queen and minor piece count, affecting the king table and safety component.

2.3. Phase 2: Neural Network Evaluation

Board Encoding

Each position is encoded as a 773-dimensional binary vector: 768 features for piece placement (12 piece types $\times$ 64 squares), 1 for side to move, and 4 for castling rights. This encoding is lossless for evaluation purposes.

Network Architecture

We use a fully connected MLP with decreasing hidden layer sizes:

$$773 \xrightarrow{\text{Linear}} 512 \xrightarrow{\text{BN+ReLU+Drop}} 256 \xrightarrow{\text{BN+ReLU+Drop}} 128 \xrightarrow{\text{BN+ReLU+Drop}} 1 \xrightarrow{\tanh} [-1, +1] \tag{1}$$

Batch normalization (BN) stabilizes training; Dropout at rate 0.2 regularizes the approximately 560,000 trainable parameters. The tanh output maps $+1$ to a White win and -1 to a Black win.

Training Data and Labeling

We extract positions from the Lichess Elite Database (December 2022), which contains games by 2400+ Elo players [4]. From 30,000 games we sample 10 middlegame positions each (skipping the first 6 moves per side), yielding 298,000 positions. We compare two labeling strategies. Game outcome labels assign the game result ($+1/0/-1$) to every extracted position. This is computationally free but noisy: a position that is winning for White may still receive label -1 if the player later blunders. Stockfish position labels evaluate each position individually with Stockfish 17 [5] at depth 8 (50 ms per position), normalizing centipawns via:

$$\text{label} = \tanh\left(\frac{cp}{400}\right) \tag{2}$$

This maps a one-pawn advantage to ± 0.24, a piece advantage to ± 0.64, and a dominant advantage to $\approx \pm 0.90$.

Training Procedure

We minimize MSE loss with Adam ($\text{lr} = 0.001$) and a ReduceLROnPlateau scheduler (factor 0.5, patience 3 epochs). We use a 90/10 train/validation split, batch size 512, for 30 epochs, retaining the best validation checkpoint. The full pipeline uses python-chess [6] and

PyTorch [7], completing in under two hours on a Ryzen 7 4000 / NVIDIA RTX 3050.

Hybrid Evaluation

The pure NN occasionally misjudges positions where a piece is directly capturable. We blend the NN score with material counting as a tactical safety net, testing three ratios:

- 70/30: $0.7 \times (\text{NN} \times 1000) + 0.3 \times \text{material}$
- 60/20/20: $0.6 \times (\text{NN} \times 1000) + 0.2 \times \text{material} + 0.2 \times \text{PST}$
- 50/50: $0.5 \times (\text{NN} \times 1000) + 0.5 \times \text{material}$

2.4. User Interface

We built a Streamlit web interface allowing a human player to compete against either engine. It provides a visual chessboard with highlighted last moves and check indicators, a real-time centipawn evaluation bar, a move history panel, and a sidebar for engine selection, search depth control, and heuristic evaluation breakdown. Figure 1 shows the interface at the start of a game.

2.5. Evaluation Methodology

Engine strength is measured through head-to-head matches using compare_engines.py: 6-game series at depth 3 with a 5-second time limit per move, alternating colors. Scoring is 1 point for a win and 0.5 for a draw. NN training quality is tracked by validation loss on held-out positions throughout training.

3. RESULTS

3.1. Neural Network Training

Table 1 summarizes training outcomes for each labeling strategy.

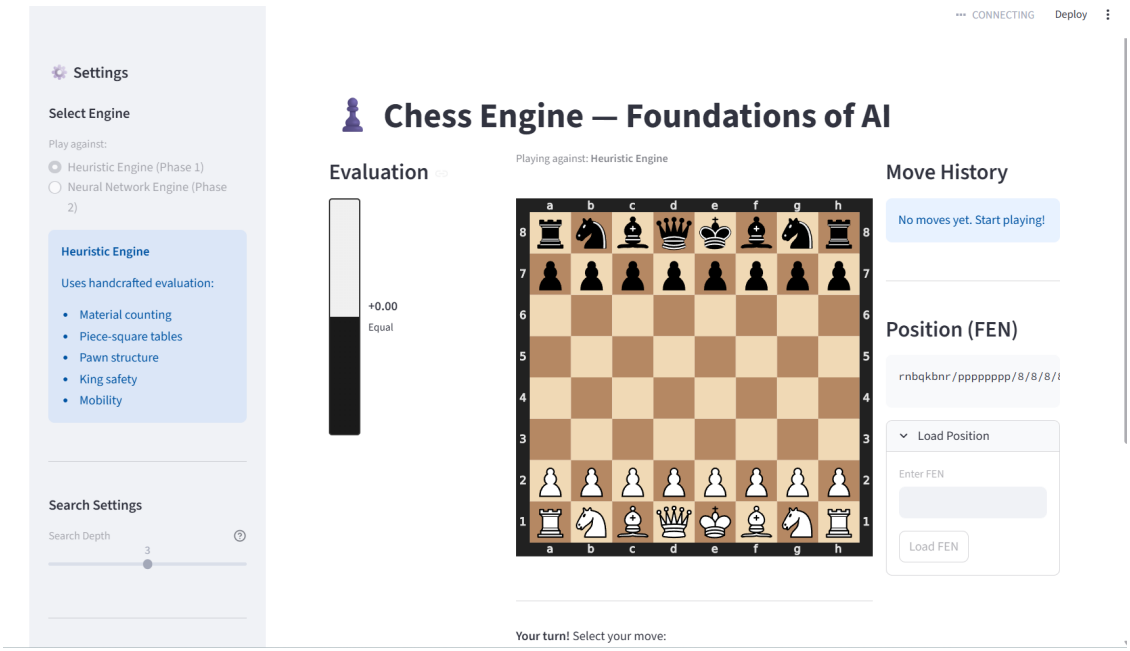

Figure 1: Streamlit web interface showing the chessboard, real-time evaluation bar, move history panel, and engine settings sidebar. The left panel allows switching between the heuristic and neural network engines and adjusting search depth.

Table 1: Training outcomes for each labeling strategy

Label Strategy	Best Val. Loss	Material Recognition
Game Outcome Labels	0.73	Failed
Stockfish Labels (Depth 8)	0.0895	Passed

With game outcome labels the model converges to 0.73 and cannot reliably identify that a position with White up a queen is favorable. It seems the model only learns the basic idea that equal positions are common, without picking up detailed features specific to each position. With Stockfish labels the model reaches a best validation loss of 0.0895, an 8× improvement, correctly ranking test positions by material advantage and assigning near-zero scores to equal openings such as the Sicilian Defense after 1.e4 c5. Figure 2 shows the training and validation loss curves over all 30 epochs.

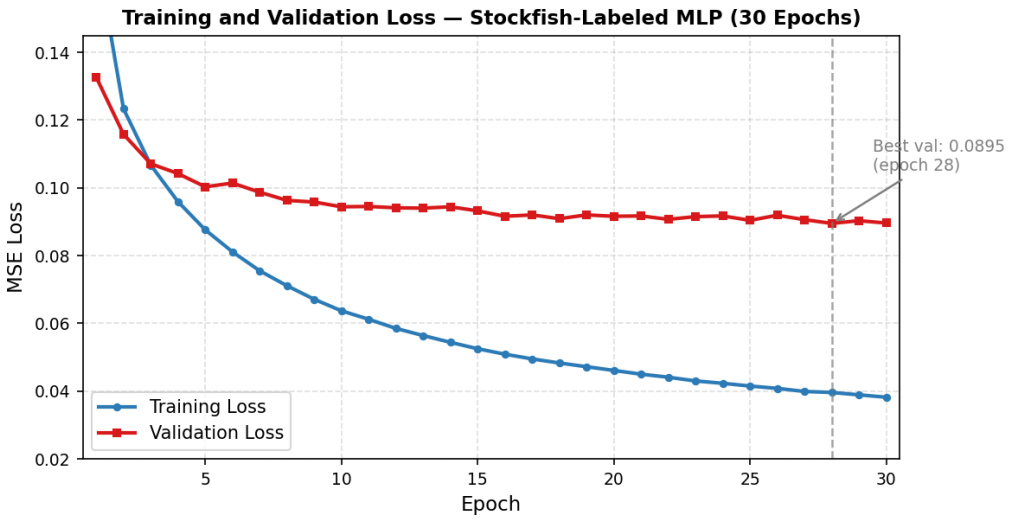

Figure 2: Training and validation loss over 30 epochs using Stockfish position labels. Validation loss reaches a best of 0.0895 at epoch 28. The gap between training and validation loss indicates that additional training data would likely yield further improvement.

3.2. Engine Match Results

Table 2 summarizes all experimental configurations. All matches are 6-game series at depth 3, alternating colors.

Table 2: Match results: Heuristic (H) vs. NN engine across all configurations

NN Configuration	H Wins	NN Wins	Draws	NN Score
Game outcome labels	6	0	0	0.0/6
Stockfish labels (pure NN)	6	0	0	0.0/6
70/30 NN + Material (Run 1)	4	0	2	1.0/6
60/20/20 NN + Mat + PST	6	0	0	0.0/6
70/30 NN + Material (Run 2)	3	0	3	1.5/6

The game-outcome network loses every game in under 35 moves. Stockfish labels fix evaluation accuracy but the pure NN still loses all 6 games due to occasional tactical misjudgments. The 70/30 hybrid produces the first draws (1.0/6, then 1.5/6 in a second run). Adding PST to the blend (60/20/20) regresses to 0/6 — the additional heuristic signal conflicts with patterns the NN already learned from Stockfish labels. Reducing NN weight to 50/50 also underperforms relative to 70/30. Table 3 shows the game-by-game breakdown for the best

configuration.

Table 3: Final match game-by-game: 70/30 hybrid NN vs. Heuristic engine, depth 3

Game	White	Black	Result	Moves
1	Heuristic	Neural Net	Draw	160
2	Neural Net	Heuristic	Heuristic wins	48
3	Heuristic	Neural Net	Draw	160
4	Neural Net	Heuristic	Heuristic wins	54
5	Heuristic	Neural Net	Draw	160
6	Neural Net	Heuristic	Heuristic wins	54

The final score is Heuristic 4.5/6, Neural Network 1.5/6. A clear pattern holds: the NN draws all three games as Black but loses all three as White. As Black it defends reactively — holding material and avoiding blunders across 160-move games. As White it must generate initiative but instead plays passive openings (1.e3 rather than 1.e4 or 1.d4), creates no central tension, and falls into inferior positions. Figure 3 shows the NN score progression across all configurations.

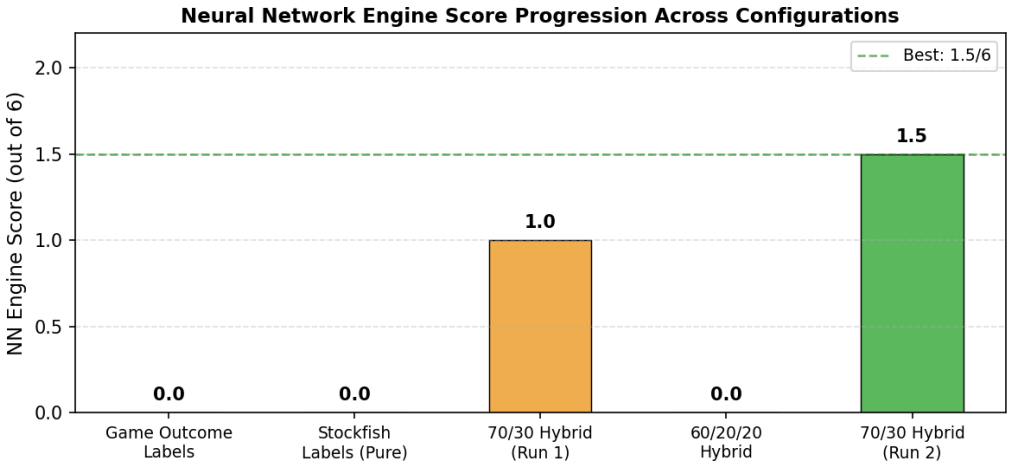

Figure 3: NN engine score progression across experimental configurations (out of 6 points). The 70/30 NN + Material hybrid (Run 2) achieves the best result at 1.5/6, a meaningful improvement from the 0/6 game-outcome baseline.

4. CONCLUSIONS

This project explored how much of the performance gap between handcrafted heuristics and learned evaluation can be closed with 298,000 training positions on consumer hardware. The heuristic engine wins 3 and draws 3 of 6 games against our best NN configuration, confirming that decades of hand-encoded chess theory remains difficult to replicate at this data scale. AlphaZero [2] closed this gap with hundreds of millions of self-play games; Stockfish NNUE with billions of labeled positions. At 298,000 positions we are orders of magnitude below both thresholds.

Three design choices drove essentially all of our improvement. First, switching to per-position Stockfish evaluations reduced validation loss 8× and produced the first tactically aware network. Game results are an extremely noisy proxy for intermediate position value; Stockfish provides clean, position-specific ground truth. Second, the 70/30 hybrid introduced a material counting safety net that prevented tactical blunders without suppressing the NN's positional contribution. Third, keeping the hybrid simple was critical — adding PST weights (60/20/20) hurt rather than helped, because the NN had already internalized some positional patterns from Stockfish labels and the additional signal created conflicting rather than complementary incentives.

The NN's most persistent weakness is asymmetric color performance: it draws as Black through reactive defense but loses as White for want of initiative. This reflects a fundamental limit of supervised evaluation learning — Stockfish labels teach the network to judge who is winning in a given position, but not which positions to steer toward. Without self-play exposure to attacking patterns, the NN defaults to locally neutral but strategically passive moves from the opening.

Future work with the highest expected impact includes scaling training data to several million positions, replacing the MLP with a CNN or transformer that models spatial board structure

more naturally, and implementing self-play reinforcement learning to teach the NN to create advantages rather than merely recognize them. An opening book for the first 10–15 moves would directly eliminate the NN's most predictable weakness. Replacing the FEN-based transposition table with Zobrist hashing would also allow the NN engine to search to the same depth as the heuristic within the same time budget, removing a structural disadvantage in tactical positions.

REFERENCES

[1] Claude E. Shannon. Programming a computer for playing chess. Philosophical Magazine, 41(314):256–275, 1950. doi: 10.1080/14786445008521796.

[2] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.

[3] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. Artificial Intelligence, 6(4):293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.

[4] Lichess.org. Lichess elite database. https://database.nikonoel.fr/, 2022. Accessed March 2026.

[5] Stockfish Development Team. Stockfish chess engine, version 17. https://stockfishchess.org, 2023. Accessed March 2026.

[6] Niklas Fiekas. python-chess: A chess library for Python. https://python-chess.readthedocs.io, 2023. Accessed February 2026.

[7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison,

Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems, volume 32, pages 8024–8035, 2019.